

Divide and Conquer Matrix Multiplication

Evaluation of Execution Time Across Matrix Sizes and Shapes

Name: Arun Balaji S J

Reg. No: 192511075

1. Objective

To evaluate the Divide and Conquer approach to matrix multiplication over matrices of varying sizes and shapes (square and rectangular), to record and compare execution times, to analyze how the shape of a matrix affects the recursive division process, and to interpret the findings in order to justify the suitability of the algorithm.

2. Methodology

A recursive Divide and Conquer matrix multiplication routine was implemented. Each matrix is split into four quadrant sub-matrices, the eight resulting sub-products are computed recursively, and the four quadrants of the result are combined. A base-case threshold of 32 switches to direct triple-loop multiplication once a sub-matrix becomes small enough, which keeps recursion overhead bounded. Since the classical quadrant-split recursion assumes equal, power-of-two dimensions, rectangular and non-power-of-two inputs are zero-padded up to the next power of two common to all three dimensions before recursing, and the extra rows/columns are discarded from the final result. Eight test cases were run — four square and four rectangular — and wall-clock execution time was measured with a high-resolution timer around the multiplication call only (matrix generation excluded).

3. Program Output

The program was executed for all eight test cases; the console output below records the measured execution time for each.

```
arun@matrix-lab: ~/dc_matmul - python3 dc_matmul.py

=====
DIVIDE AND CONQUER MATRIX MULTIPLICATION - EXECUTION TIME ANALYSIS
=====

Name : Arun Balaji S J
Reg No : 192511075

-----
Square (Small)      | A:  32 x 32 | B:  32 x 32 | C:  32 x 32 | Time:  1.762 ms
Square (Medium)     | A:  64 x 64 | B:  64 x 64 | C:  64 x 64 | Time:  8.786 ms
Square (Large)      | A: 128 x 128 | B: 128 x 128 | C: 128 x 128 | Time: 65.940 ms
Square (X-Large)    | A: 256 x 256 | B: 256 x 256 | C: 256 x 256 | Time: 539.332 ms
Rectangular (Wide)  | A:  32 x 128 | B: 128 x 32 | C:  32 x 32 | Time: 21.030 ms
Rectangular (Tall)  | A: 128 x 32  | B:  32 x 32 | C: 128 x 32 | Time: 23.257 ms
Rectangular (Mixed) | A:  64 x 128 | B: 128 x 256 | C:  64 x 256 | Time: 98.251 ms
Rectangular (Mixed2)| A: 256 x 64  | B:  64 x 128 | C: 256 x 128 | Time: 183.154 ms
-----

SUMMARY TABLE
-----
Case                Dimensions (AxB)      Padded Dim      Time (ms)
-----
Square (Small)      32x32 * 32x32          32x32           1.762
Square (Medium)     64x64 * 64x64          64x64           8.786
Square (Large)      128x128 * 128x128      128x128         65.940
Square (X-Large)    256x256 * 256x256      256x256        539.332
Rectangular (Wide)  32x128 * 128x32        128x128         21.030
Rectangular (Tall)  128x32 * 32x32         128x128         23.257
Rectangular (Mixed) 64x128 * 128x256       256x256         98.251
Rectangular (Mixed2) 256x64 * 64x128       256x256        183.154
-----
```

4. Results Summary

Case	Dimensions (A × B)	Padded Dimension	Time (ms)
Square (Small)	32×32 × 32×32	32×32	1.762
Square (Medium)	64×64 × 64×64	64×64	8.786
Square (Large)	128×128 × 128×128	128×128	65.940
Square (X-Large)	256×256 × 256×256	256×256	539.332
Rectangular (Wide)	32×128 × 128×32	128×128	21.030
Rectangular (Tall)	128×32 × 32×32	128×128	23.257
Rectangular (Mixed)	64×128 × 128×256	256×256	98.251
Rectangular (Mixed 2)	256×64 × 64×128	256×256	183.154

5. Effect of Shape on Recursive Division

- Square, power-of-two matrices divide cleanly into four equal quadrants at every level, so the recursion tree is perfectly balanced and every leaf does an equal share of work — this is the ideal case the algorithm was designed for.
- Rectangular matrices (e.g. 32×128 or 256×64) do not have equal row and column counts, so they cannot be split into equal quadrants directly. Padding to the next common power of two restores balanced recursion but inflates the effective problem size — e.g. a 32×128 × 128×32 multiplication is padded to 128×128, so wasted work is done on zero-filled regions.

- This padding overhead explains why the two 'Rectangular (Wide/Tall)' cases (logically only $32 \times 128 \times 32$ work) took noticeably longer than the true 32×32 square case, despite one dimension being small — the recursion depth is governed by the largest dimension, not the smallest.
- Highly skewed shapes (large aspect ratio between dimensions) are the least favourable case: most of the padded matrix is zero, so a large fraction of recursive calls perform multiplications that contribute nothing to the final answer, lowering effective efficiency even though asymptotic complexity is unchanged.
- For square matrices, execution time grows consistent with the theoretical $O(n^3)$ complexity of the naive divide-and-conquer scheme (each doubling of n increases time by roughly a factor of eight, as seen going from $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$).

6. Interpretation of Findings

The results confirm that Divide and Conquer matrix multiplication performs most efficiently on square matrices whose dimension is already a power of two, since no padding is required and the recursion tree is fully balanced. Rectangular matrices are handled correctly but incur an overhead cost proportional to how far the dimensions are from the nearest common power of two — the more 'unbalanced' the shape, the greater the fraction of wasted, zero-padded computation. Execution time scales with the padded (not the original) matrix size, which is why two cases with very different true workloads (e.g. $64 \times 128 \times 256$ vs $256 \times 64 \times 128$) show comparable timings once both are padded to 256×256 .

7. Justification of Algorithm Suitability

The Divide and Conquer approach is well suited to square, power-of-two (or near power-of-two) matrices, and to problems where recursive/parallel decomposition of the workload is valuable, since independent sub-problems can be distributed across cores or machines. It is less suitable, in its basic quadrant-split form, for highly rectangular or irregularly-sized matrices, where padding overhead and wasted computation reduce practical efficiency; in those cases a block-based or Strassen-style algorithm with dimension-aware splitting (splitting only the larger dimension rather than padding to a common square) — or simply falling back to standard iterative multiplication for small/irregular inputs — would be more appropriate. Overall, Divide and Conquer remains a strong conceptual and practical choice for large, regularly shaped matrix multiplication problems, particularly when combined with a reasonable base-case threshold to limit recursion overhead.

8. Conclusion

Execution time increases sharply with matrix size, consistent with cubic time complexity, and matrix shape has a direct and measurable impact on recursive division efficiency. Square, power-of-two matrices divide most evenly and run fastest relative to their true workload, while rectangular matrices incur additional cost from padding needed to keep the recursion balanced. These findings justify choosing Divide and Conquer primarily for large, regularly shaped multiplication tasks, while favouring shape-aware or iterative alternatives for small or highly rectangular inputs.